ACM AI PROJECTS

Complete AI & Machine Learning Reference Guide

Beginner-Friendly Edition

For ACM AI project members, project leads, and anyone building their first end-to-end AI project

Purpose: Use this guide as a shared team reference. It explains the core ideas behind data science, machine learning, neural networks, computer vision, natural language processing, generative AI, collaboration, debugging, and deployment in beginner-friendly language.

Created for ACM AI Projects

Table of Contents

Tip: In Google Docs, apply the built-in Table of Contents feature after import if you want clickable page numbers. All section titles already use heading styles.

1. How to Use This Guide

This handbook is written for students who may be joining an AI project with little or no prior experience. You do not need to memorize every section before contributing. Start with the project workflow, Python basics, data preprocessing, and the specific model family your team is using.

What every project member should understand first

- **The problem:** What are we predicting, generating, detecting, ranking, or understanding?
- **The data:** Where does it come from, what does each row represent, and what could be wrong with it?
- **The baseline:** What simple method should a more advanced model beat?
- **The metric:** How will the team decide whether the model is actually useful?
- **The workflow:** How do data, code, experiments, documentation, and GitHub fit together?

Suggested reading order

1. Read Sections 2, 7, 8, 14, and 24 for the shared foundation every member needs.
2. Read Sections 9–13 if the project uses traditional machine learning.
3. Read Sections 16–18 if the project uses neural networks.
4. Read Section 19 for image projects and Sections 20–22 for text, chatbot, or LLM projects.
5. Use Sections 27–30 as quick references during implementation.

Beginner mindset: Confusion is normal. AI projects combine programming, statistics, domain knowledge, and teamwork. Ask what each step is doing and why it is needed instead of copying code without understanding it.

2. The Big Picture: AI, Machine Learning, Deep Learning, and Data Science

Artificial intelligence

Artificial intelligence (AI) is the broad goal of building systems that perform tasks associated with human intelligence, such as recognizing images, understanding language, making recommendations, planning, or generating content.

Machine learning

Machine learning (ML) is a way to build AI systems by learning patterns from data instead of writing a separate rule for every situation. A model receives examples and adjusts internal parameters so that it can make useful predictions on new data.

Deep learning

Deep learning is a subfield of machine learning based on neural networks with many layers. It is especially powerful for images, audio, text, and other unstructured data, but it often needs more data, computing power, and tuning than traditional models.

Data science

Data science is the larger process of turning data into useful knowledge or decisions. It includes collecting data, cleaning it, exploring patterns, visualizing results, building models, communicating findings, and monitoring systems after deployment.

Term	Main idea	Example
AI	The overall field of intelligent systems	A virtual assistant that understands questions
Machine learning	Learns patterns from examples	Predict whether an email is spam

Term	Main idea	Example
Deep learning	Uses multilayer neural networks	Recognize objects in photos
Data science	Uses data to answer questions and support decisions	Analyze product usage and build a churn model

Types of machine learning

Type	What the model receives	Goal	Examples
Supervised learning	Inputs with known answers (labels)	Predict an answer for new inputs	Classification, regression
Unsupervised learning	Inputs without labels	Find structure or patterns	Clustering, dimensionality reduction
Self-supervised learning	Labels created from the data itself	Learn useful representations	Masked-word prediction in language models
Reinforcement learning	Rewards from interacting with an environment	Learn actions that maximize reward	Game-playing agents, robotics

Rule of thumb: Most student project teams begin with supervised learning because the goal and evaluation are easier to define. Deep learning is not automatically better; the best model is the simplest one that solves the problem reliably.

3. Project Setup and Essential Tools

Recommended tools

Tool	Purpose	What beginners should know
Python	Primary programming language	Variables, functions, loops, imports, errors
Jupyter Notebook / Google Colab	Interactive experiments	Run cells in order and restart the runtime when state becomes confusing
VS Code	Editing larger projects	Folders, terminal, extensions, debugging
Git and GitHub	Version control and collaboration	Clone, branch, commit, pull, push, pull request
NumPy	Numerical arrays	Shapes, indexing, vectorized operations
pandas	Tables and data cleaning	DataFrames, filtering, groupby, missing values
Matplotlib	Visualization	Plot distributions, relationships, and evaluation results
scikit-learn	Traditional ML	Preprocessing, models, metrics, pipelines
PyTorch or TensorFlow	Deep learning	Tensors, models, losses, optimizers, training loops

Environment setup

A Python environment keeps each project's packages separate. This prevents one project from breaking because another project needs a different package version.

Typical local setup

```
# Create a virtual environment
python -m venv .venv

# Activate on macOS/Linux
source .venv/bin/activate

# Activate on Windows PowerShell
.venv\Scripts\Activate.ps1

# Install common packages
pip install numpy pandas matplotlib scikit-learn jupyter
```

requirements.txt

A requirements file records the packages needed to run a project. Team members can reproduce the environment with one command.

```
pip freeze > requirements.txt
pip install -r requirements.txt
```

Team rule: Do not commit private API keys, passwords, credentials, or large raw datasets to GitHub. Use environment variables, a .env file excluded by .gitignore, and approved shared storage.

4. Python Foundations for AI Projects

Core data types

Type	Example	Use
int / float	7, 3.14	Numbers
str	"cat"	Text
bool	True	Conditions
list	[1, 2, 3]	Ordered collection
tuple	(28, 28, 3)	Fixed collection, often shapes
dict	{"lr": 0.001}	Key-value settings
set	{"cat", "dog"}	Unique values

Variables, conditions, loops, and functions

```
scores = [0.71, 0.84, 0.79]

best_score = max(scores)
if best_score >= 0.80:
    print("Promising model")
else:
    print("Needs improvement")

for i, score in enumerate(scores):
    print(i, score)

def accuracy(correct, total):
    if total == 0:
        return 0.0
    return correct / total
```

List comprehensions

List comprehensions are a compact way to transform or filter values. Read them as “create this output for each item that meets the condition.”

```
values = [1, 2, 3, 4, 5]
squares = [x ** 2 for x in values]
even_squares = [x ** 2 for x in values if x % 2 == 0]
```

Imports and modules

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
```

Errors and debugging

Error	Usually means	First thing to check
NameError	A variable is not defined	Spelling and whether the cell ran
TypeError	An operation received the wrong type	Use type(variable)
KeyError	A dictionary key or DataFrame column is missing	Print available keys/columns
IndexError	An index is outside the valid range	Check len() or array shape
ValueError	A value has the wrong form or shape	Inspect values and dimensions
ModuleNotFoundError	Package is not installed in the active environment	Activate the correct environment and install it

Good practice: Write small functions with clear inputs and outputs. Avoid one notebook cell that performs the entire project. Small pieces are easier to test, review, and reuse.

5. NumPy, pandas, and Data Visualization

NumPy arrays

NumPy arrays store numerical data efficiently. Unlike Python lists, they support fast vectorized operations that apply to many values at once.

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
print(x.shape)        # (3,)
print(x.mean())       # 2.0
print((x - x.mean()) / x.std())  # standardized values
```

Shape is one of the most important ideas

- **Scalar:** one number, shape ()
- **Vector:** one-dimensional list of numbers, shape (n,)
- **Matrix:** rows and columns, shape (n, d)

- **Image batch:** often shape (batch, channels, height, width) or (batch, height, width, channels)

pandas DataFrames

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
print(df.shape)
print(df.dtypes)
print(df.isna().sum())

# Select rows and columns
features = df[["age", "income"]]
adults = df[df["age"] >= 18]

# Group and summarize
summary = df.groupby("category")["score"].mean()
```

Useful exploration questions

- How many rows and columns are there?
- What does one row represent?
- Which columns are numerical, categorical, text, image paths, IDs, or labels?
- Are there missing values, duplicates, impossible values, or extreme outliers?
- Is the target balanced?
- Could the same person, user, or item appear in both training and test data?

Visualization

```
import matplotlib.pyplot as plt

# Distribution
plt.hist(df["age"].dropna(), bins=20)
plt.xlabel("Age")
plt.ylabel("Count")
plt.title("Age Distribution")
plt.show()

# Relationship
plt.scatter(df["feature"], df["target"], alpha=0.5)
plt.xlabel("Feature")
plt.ylabel("Target")
plt.show()
```

Why visualize?: Summary statistics can hide data problems. A plot may reveal skew, clusters, class imbalance, impossible values, or data leakage before modeling begins.

6. Math Foundations You Actually Need

Vectors and matrices

A vector is an ordered list of numbers. A feature vector might describe one example: [age, income, number_of_purchases]. A matrix is a table of vectors. In supervised learning, X usually represents the feature matrix and y represents the target values.

Dot product

The dot product multiplies matching elements and adds the results. Linear models use it to combine features with learned weights: prediction = w·x + b.

```
# x = [2, 3], w = [0.5, -1]
# dot product = 2(0.5) + 3(-1) = -2
```

Mean, variance, and standard deviation

- **Mean:** the average value.
- **Variance:** the average squared distance from the mean.
- **Standard deviation:** the typical scale of variation, equal to the square root of variance.

Probability and conditional probability

Probability measures uncertainty. Conditional probability P(A|B) means the probability of A given that B has occurred. Classification models often estimate a conditional probability such as P(spam | email features).

Derivatives and gradients

A derivative measures how quickly a quantity changes. A gradient is a vector of derivatives, one for each model parameter. It tells us which direction increases the loss most quickly. Gradient descent moves in the opposite direction to reduce the loss.

Logarithms

Logarithms appear in cross-entropy loss, information theory, and transformations of skewed data. They turn multiplication into addition and strongly penalize confident wrong predictions in classification.

Distance and similarity

Measure	Meaning	Common use
Euclidean distance	Straight-line distance	k-nearest neighbors, clustering
Manhattan distance	Sum of absolute differences	Robust distance in grid-like settings
Cosine similarity	Similarity of direction rather than magnitude	Text embeddings, semantic search
Dot product similarity	Alignment influenced by magnitude	Neural networks and retrieval systems

Do not panic about math: You can contribute before mastering every derivation. Focus first on what each quantity means, how changing it affects the model, and how to recognize when the model is behaving badly.

7. The End-to-End Machine Learning Workflow

6. Define the problem and the user need.
7. Choose the unit of analysis: what does one example represent?
8. Collect or access the data legally and ethically.
9. Explore and clean the data.
10. Define features, labels, and a train/validation/test split.
11. Create a simple baseline.
12. Train candidate models.
13. Evaluate with appropriate metrics and error analysis.
14. Iterate on data, features, model choices, and hyperparameters.
15. Document results, limitations, and next steps.

16. Deploy only when the model is useful, safe, and reproducible.
17. Monitor performance and data changes after deployment.

Problem framing

Question	Example
What is the input?	A product review
What is the output?	Positive, neutral, or negative sentiment
Who uses the output?	A product analytics team
What action follows?	Prioritize issues and summarize feedback
What is the cost of a mistake?	A negative review may be overlooked
What metric matters?	Macro F1 because all classes matter

Always build a baseline

A baseline is a simple reference point. For classification, it might always predict the majority class. For regression, it might predict the training mean. For text, it might use TF-IDF with logistic regression before trying a transformer. A complex model that does not beat a reasonable baseline is not helping.

Avoid data leakage

Data leakage occurs when information unavailable at prediction time accidentally enters training. Leakage can produce excellent validation scores and terrible real-world performance.

- Fitting a scaler on the full dataset before splitting.
- Using a feature created after the outcome occurred.
- Allowing the same user, patient, document, or near-duplicate image in train and test.
- Selecting features using the test set.
- Tuning repeatedly against the final test set.

Golden rule: The test set is for the final unbiased evaluation. Use the validation set or cross-validation for model selection and tuning.

8. Data Cleaning, Preprocessing, and Feature Engineering

Missing values

Strategy	When it may work	Risk
Drop rows	Few rows are missing and missingness is random	Loses data and may introduce bias
Mean/median imputation	Numerical columns	Reduces variation; mean is sensitive to outliers
Most-frequent imputation	Categorical columns	May overrepresent the most common class
Add a missing indicator	Missingness may itself carry information	Can encode undesirable process artifacts
Model-based imputation	Complex patterns and enough data	Adds complexity and potential leakage

Categorical variables

- **One-hot encoding:** creates one binary column per category; strong default for low-cardinality categories.
- **Ordinal encoding:** maps ordered categories to numbers; use only when order is meaningful.
- **Target encoding:** uses target statistics; powerful but must be performed inside training folds to avoid leakage.
- **Embeddings:** learn dense representations; useful for many categories or deep-learning systems.

Scaling

Method	What it does	Often useful for
Standardization	Centers at 0 and scales to standard deviation 1	Linear models, SVM, kNN, neural networks
Min-max scaling	Maps values to a fixed range such as 0 to 1	Neural networks and bounded input requirements
Robust scaling	Uses median and interquartile range	Data with strong outliers
No scaling	Keeps original units	Trees and tree ensembles usually do not require scaling

Feature engineering

Feature engineering creates useful inputs from raw data. Examples include extracting day of week from a timestamp, computing price per unit, counting words, aggregating a user's recent activity, or creating domain-specific ratios.

Use pipelines

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_columns),
    ("cat", categorical_pipe, categorical_columns)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

Why pipelines matter: A pipeline ensures each preprocessing step is learned only from training data and applied consistently to validation, test, and future production data.

9. Supervised Learning Fundamentals

Features and labels

Features are the inputs the model uses. The label or target is the answer the model learns to predict. The feature matrix is commonly called X, and the target vector is called y.

Regression vs. classification

Problem	Output	Examples
Regression	A continuous number	Price, temperature, time, demand
Binary classification	One of two classes	Fraud/not fraud, spam/not spam
Multiclass classification	One of several classes	Animal species, topic category
Multilabel classification	Zero or more labels at once	Image tags, document topics

Parameters vs. hyperparameters

Parameters are learned from data, such as regression coefficients or neural-network weights. Hyperparameters are chosen by the team, such as tree depth, learning rate, number of neighbors, or batch size.

Training, validation, and test sets

Split	Purpose
Training set	Fit model parameters
Validation set	Choose models and hyperparameters
Test set	Estimate final performance on unseen data

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Stratification: For classification, stratify=y helps preserve class proportions in each split. For grouped or time-dependent data, use a group-aware or time-based split instead of a random split.

10. Regression Models

Linear regression

Linear regression predicts a number using a weighted sum of features. It is fast, interpretable, and a strong baseline when relationships are roughly linear.

Model idea: prediction = w1x1 + w2x2 + ... + b. Each coefficient represents how the prediction changes when that feature increases by one unit while other features stay fixed, assuming the model is correctly specified.

Strengths	Weaknesses
Fast and interpretable	Misses complex nonlinear relationships
Works well with limited data	Sensitive to multicollinearity and outliers

Strengths	Weaknesses
Easy baseline	Assumptions may not hold

Regularized linear models

Model	Penalty	Effect
Ridge	Squared coefficient sizes (L2)	Shrinks coefficients smoothly; helps correlated features
Lasso	Absolute coefficient sizes (L1)	Can set some coefficients to exactly zero
Elastic Net	Combination of L1 and L2	Balances sparsity and stability

Polynomial regression

Polynomial features allow a linear model to represent curves, but high-degree polynomials can overfit and behave unpredictably outside the training range.

Tree-based regression

Decision trees, random forests, and gradient boosting can capture nonlinear relationships and interactions with less manual feature design. They are often strong choices for structured tabular data.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

model = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(mean_absolute_error(y_test, pred))
```

Regression metrics

Metric	Interpretation	Watch out for
MAE	Average absolute error in target units	Treats all errors linearly
MSE	Average squared error	Large errors dominate
RMSE	Square root of MSE; target units	Still sensitive to large errors
R^2	Fraction of variance explained relative to mean baseline	Can be negative; does not show error size

11. Classification Models

Logistic regression

Logistic regression is a linear classifier that estimates class probabilities. It is fast, interpretable, and often surprisingly strong for tabular and sparse text data.

k-nearest neighbors (kNN)

kNN predicts based on the labels of nearby training examples. It is intuitive but becomes slow at prediction time, is sensitive to feature scaling, and can struggle in high-dimensional spaces.

Support vector machines (SVMs)

An SVM finds a boundary with a large margin between classes. Kernel SVMs can model nonlinear boundaries, but they may be slow on large datasets and require careful scaling and tuning.

Naive Bayes

Naive Bayes uses probability rules and a strong independence assumption. Despite the simplified assumption, it is fast and can work well for text classification and small datasets.

Decision trees

Decision trees repeatedly split the data using feature thresholds. They are easy to visualize but can overfit if allowed to grow too deep.

Model	Good starting use	Important preprocessing
Logistic regression	Interpretable baseline, text, tabular classification	Scale numeric inputs; encode categories
kNN	Small datasets with meaningful distances	Scale all features
SVM	Medium-sized, high-dimensional data	Scale features; tune C and kernel
Naive Bayes	Text or quick probabilistic baseline	Represent text as counts or TF-IDF
Decision tree	Interpretable nonlinear rules	Little scaling needed

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

model = LogisticRegression(max_iter=1000, class_weight="balanced")
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(classification_report(y_test, pred))
```

12. Decision Trees, Random Forests, and Gradient Boosting

Decision trees

A decision tree asks a sequence of questions such as “Is age < 30?” Each split tries to make the resulting groups more pure with respect to the target. Trees naturally model nonlinear patterns and interactions.

- **max_depth:** limits how many levels the tree can grow.
- **min_samples_split:** minimum samples needed to split a node.
- **min_samples_leaf:** minimum samples allowed in a final leaf.
- **criterion:** the rule used to judge split quality, such as Gini impurity or entropy.

Random forests

A random forest trains many trees on bootstrapped samples and random subsets of features, then averages their predictions. Combining many different trees reduces variance and usually generalizes better than one tree.

Gradient boosting

Gradient boosting builds trees sequentially. Each new tree focuses on correcting the errors of the current ensemble. This often produces excellent tabular-data performance, but it can overfit if the learning rate is too large or too many trees are added.

Method	How trees are combined	Typical behavior
Single tree	One tree	Interpretable but unstable and prone to overfitting
Random forest	Many independent-ish trees averaged	Robust, strong default, less tuning
Gradient boosting	Trees added sequentially to fix errors	Often higher accuracy, more sensitive to tuning

Key gradient boosting hyperparameters

- **n_estimators:** number of boosting stages or trees.
- **learning_rate:** how strongly each new tree changes the model.
- **max_depth / max_leaf_nodes:** complexity of each tree.
- **subsample:** fraction of rows used per stage; values below 1 add randomness.
- **regularization:** controls complexity and prevents overfitting.

Learning-rate tradeoff: A smaller learning rate usually needs more trees but can generalize better. Do not tune n_estimators without considering learning_rate.

13. Unsupervised Learning

Clustering

Clustering groups examples without known labels. Clusters are not automatically meaningful categories; the team must inspect and interpret them using domain knowledge.

k-means

k-means assigns each point to the nearest cluster center and repeatedly updates the centers. It works best for roughly spherical, similarly sized clusters and requires choosing k.

Hierarchical clustering

Hierarchical clustering builds a tree of nested groups. A dendrogram helps visualize how clusters merge, but the method may be expensive for large datasets.

DBSCAN

DBSCAN identifies dense regions and marks isolated points as noise. It can discover irregularly shaped clusters and does not require a fixed number of clusters, but its distance parameters can be difficult to tune.

Dimensionality reduction

Method	Main idea	Use
PCA	Linear directions of maximum variance	Compression, denoising, visualization, removing correlation
t-SNE	Preserves local neighborhoods for visualization	Exploratory 2D/3D plots; not a general feature transform
UMAP	Graph-based neighborhood preservation	Visualization and sometimes preprocessing

Method	Main idea	Use
Autoencoder	Neural network compresses and reconstructs data	Nonlinear representation learning

Visualization warning: Distances and cluster shapes in a 2D t-SNE or UMAP plot can be misleading. Use these plots for exploration, not as proof that true classes exist.

14. Model Evaluation and Metrics

Confusion matrix terms

Term	Meaning
True positive (TP)	Predicted positive and actually positive
True negative (TN)	Predicted negative and actually negative
False positive (FP)	Predicted positive but actually negative
False negative (FN)	Predicted negative but actually positive

Classification metrics

Metric	Question it answers	Use when
Accuracy	What fraction were correct?	Classes are balanced and mistakes have similar costs
Precision	Of predicted positives, how many were truly positive?	False positives are costly
Recall	Of actual positives, how many did we find?	False negatives are costly
F1 score	How well are precision and recall balanced?	Both error types matter
ROC-AUC	How well are positives ranked above negatives across thresholds?	Comparing binary ranking quality
PR-AUC	How well does precision-recall trade off?	Positive class is rare
Log loss	How accurate and calibrated are predicted probabilities?	Probability quality matters

Macro, micro, and weighted averaging

- **Macro average:** compute the metric per class and average equally; emphasizes minority classes.
- **Micro average:** pool all decisions before computing the metric; emphasizes common classes.
- **Weighted average:** average per-class metrics weighted by class frequency.

Thresholds

A classifier may output a probability, and the team chooses a threshold to convert it into a class. The default threshold of 0.5 is not always appropriate. Select a threshold based on the real cost of false positives and false negatives using validation data.

Cross-validation

Cross-validation repeatedly trains on different subsets and validates on the remaining fold. It gives a more stable performance estimate when data is limited.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    model, X, y,
    cv=5,
    scoring="f1_macro"
)
print(scores.mean(), scores.std())
```

Error analysis

Metrics summarize performance, but error analysis explains it. Inspect incorrect examples, group errors by class or user segment, look for mislabeled data, and identify recurring failure patterns. Often the next improvement comes from better data rather than a more complex model.

15. Overfitting, Underfitting, Bias, and Variance

Underfitting

A model underfits when it is too simple or insufficiently trained to capture the pattern. Training and validation performance are both poor.

Overfitting

A model overfits when it memorizes training-specific noise. Training performance is very strong, but validation performance is much worse.

Pattern	Training performance	Validation performance	Possible response
Underfitting	Poor	Poor	Use better features, a more flexible model, or train longer
Good fit	Strong	Strong and close to training	Evaluate carefully and perform error analysis
Overfitting	Very strong	Noticeably worse	Regularize, simplify, add data, augment, or stop earlier

Bias and variance

Bias is error from assumptions that are too simple. Variance is sensitivity to the exact training sample. Simple models tend to have higher bias and lower variance; very flexible models tend to have lower bias and higher variance.

Regularization

- L1/L2 penalties for linear models and neural networks.
- Limiting tree depth or minimum leaf size.
- Dropout in neural networks.
- Data augmentation for images or text.
- Early stopping based on validation loss.
- Collecting more diverse and representative data.

Diagnosis first: Do not apply random fixes. Compare training and validation curves to determine whether the main problem is underfitting, overfitting, optimization, data quality, or an inappropriate metric.

16. Gradient Descent and Optimization

What is a loss function?

A loss function converts model mistakes into a number. Training tries to find parameters that make the average loss small. Examples include mean squared error for regression and cross-entropy for classification.

Gradient descent intuition

Imagine standing on a foggy hill and trying to reach the lowest point. The gradient tells you the direction of steepest uphill movement, so you step in the opposite direction. Repeating this process updates model parameters toward lower loss.

Update rule: new_parameter = old_parameter − learning_rate × gradient.

Learning rate

Learning rate	What may happen
Too small	Training is extremely slow and may appear stuck
Reasonable	Loss decreases steadily
Too large	Loss oscillates, explodes, or becomes NaN

Batch, stochastic, and mini-batch gradient descent

Method	Data used per update	Tradeoff
Batch gradient descent	Entire training set	Stable but expensive
Stochastic gradient descent	One example	Noisy but frequent updates
Mini-batch gradient descent	Small batch	Standard deep-learning compromise

Common optimizers

Optimizer	Idea	Typical note
SGD	Basic gradient updates	Can generalize well but needs tuning
SGD + momentum	Accumulates movement in consistent directions	Reduces oscillation and speeds progress
Adam	Adaptive learning rates plus momentum-like estimates	Strong beginner default for many neural networks
AdamW	Adam with decoupled weight decay	Common for transformers and modern deep learning

```
# Conceptual gradient descent
parameter = 3.0
learning_rate = 0.1

for step in range(100):
    gradient = compute_gradient(parameter)
    parameter = parameter - learning_rate * gradient
```

Important: Gradient descent finds a useful minimum, not necessarily the single global minimum. In deep learning, optimization behavior depends on initialization, architecture, normalization, batch size, and learning-rate scheduling.

17. Neural Networks from the Ground Up

A neuron

A neuron computes a weighted sum of inputs, adds a bias, and passes the result through an activation function. A layer contains many neurons, and a network stacks layers to learn increasingly useful representations.

Neuron idea: output = activation(w·x + b).

Layers

- **Input layer:** receives the features.
- **Hidden layers:** learn intermediate representations.
- **Output layer:** produces the final prediction, such as a number or class probabilities.

Activation functions

Activation	Behavior	Common use
ReLU	max(0, x)	Default hidden-layer activation in many networks
Leaky ReLU	Small slope for negative inputs	Reduces “dead ReLU” problem
Sigmoid	Maps to 0–1	Binary output probability
Softmax	Converts logits into probabilities summing to 1	Multiclass output
Tanh	Maps to −1–1	Some recurrent or compact networks
GELU	Smooth gating-like activation	Transformers

Forward pass

During the forward pass, data moves through the network to produce predictions and calculate loss.

Backpropagation

Backpropagation applies the chain rule to calculate how much each parameter contributed to the loss. The optimizer then updates the parameters using those gradients.

A small PyTorch network

```
import torch
from torch import nn

class SimpleNet(nn.Module):
    def __init__(self, input_dim, num_classes):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        return self.network(x)

model = SimpleNet(input_dim=20, num_classes=3)
```

Logits: Classification networks usually output raw scores called logits. Cross-entropy loss applies the needed probability transformation internally, so do not add softmax before the loss unless the framework specifically requires it.

18. Training Neural Networks Well

Core training loop

```
model.train()
for features, labels in train_loader:
    optimizer.zero_grad()          # clear old gradients
    logits = model(features)       # forward pass
    loss = loss_fn(logits, labels)
    loss.backward()                # backpropagation
    optimizer.step()               # update parameters
```

Evaluation loop

```
model.eval()
correct = 0
with torch.no_grad():
    for features, labels in val_loader:
        logits = model(features)
        predictions = logits.argmax(dim=1)
        correct += (predictions == labels).sum().item()
```

Epochs, batches, and iterations

- **Epoch:** one complete pass through the training set.
- **Batch:** a subset processed together.
- **Iteration/step:** one parameter update using one batch.

Weight initialization

Weights should begin with small, carefully scaled random values. Standard framework layers use reasonable defaults. Initializing every weight to the same value prevents neurons from learning different features.

Normalization

Batch normalization and layer normalization stabilize activations and gradients. Batch normalization is common in CNNs; layer normalization is central to transformers.

Dropout

Dropout randomly disables some activations during training, discouraging the network from depending on any one path. It is turned off automatically in evaluation mode.

Early stopping

Save the model when validation performance improves and stop after it has failed to improve for several epochs. This avoids continuing until the model overfits.

Training checklist

- Confirm inputs and labels have the expected shapes and data types.
- Overfit a tiny batch to verify the model can learn at all.

- Plot training and validation loss.
- Check for NaN or infinite values.
- Use a reproducible random seed when comparing experiments.
- Save the best validation checkpoint, not only the final epoch.
- Inspect per-class performance and example errors.

19. Convolutional Neural Networks and Computer Vision

Why images are different

Images contain spatial structure: nearby pixels are related, patterns can appear in different locations, and raw images have many values. Convolutional neural networks (CNNs) exploit this structure.

Convolution

A convolutional filter slides across an image and computes local pattern responses. Early layers may detect edges and textures; deeper layers may represent shapes and object parts.

Important CNN terms

Term	Meaning
Kernel/filter	Small learned grid that detects a pattern
Feature map	Output produced by applying a filter
Stride	How far the filter moves each step
Padding	Extra border values used to control output size
Pooling	Downsamples feature maps
Channels	Input or feature dimensions, such as RGB = 3 channels

Output size intuition

Larger kernels see wider local regions. Larger stride reduces spatial size. Padding can preserve edge information and keep height and width from shrinking too quickly.

Basic CNN

```
class SmallCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Data augmentation

Augmentation creates realistic variations of training images, such as crops, flips, rotations, or color changes. It increases diversity without collecting entirely new images. Transformations must preserve the label; flipping text or medical images may change meaning.

Transfer learning

Transfer learning starts with a model pretrained on a large dataset and adapts it to a new task. This is often the best choice for student projects with limited labeled images.

- Freeze the pretrained backbone and train a new classifier first.
- Then optionally unfreeze later layers and fine-tune with a smaller learning rate.
- Use the preprocessing expected by the pretrained model.
- Compare against a simple baseline and report class-wise errors.

Common computer vision tasks

Task	Output
Image classification	One or more labels for an image
Object detection	Labels plus bounding boxes
Semantic segmentation	A class for every pixel
Instance segmentation	Separate mask for each object
Image generation	A new image or edited image
Optical character recognition	Text extracted from an image

20. Natural Language Processing Fundamentals

What is NLP?

Natural language processing (NLP) builds systems that work with human language. Tasks include sentiment analysis, topic classification, named-entity recognition, translation, summarization, question answering, search, and text generation.

Text preprocessing

- **Tokenization:** split text into units such as words, subwords, or characters.
- **Normalization:** standardize case or formatting when appropriate.
- **Stop-word removal:** remove common words in some traditional models; not always helpful.
- **Stemming:** reduce words to crude roots.
- **Lemmatization:** reduce words to dictionary forms.
- **Subword tokenization:** represent rare words as reusable pieces; standard for transformers.

Bag of words and TF-IDF

Bag of words represents a document by token counts and ignores word order. TF-IDF gives more weight to words that are frequent in one document but uncommon across the collection. These sparse features work well with logistic regression, linear SVMs, and Naive Bayes.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

text_model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
```

```
    ("classifier", LogisticRegression(max_iter=1000))
])
text_model.fit(train_texts, train_labels)
```

Embeddings

An embedding is a dense numerical representation in which semantically related items tend to be closer. Embeddings can represent words, sentences, documents, images, users, or products.

Sequence models

Recurrent neural networks (RNNs), LSTMs, and GRUs process sequences over time and were widely used before transformers. They remain useful for understanding sequence modeling, but transformers are now common for many language tasks.

Common NLP pitfalls

- Train/test leakage from duplicate or near-duplicate text.
- Using accuracy on highly imbalanced labels.
- Cleaning away meaningful punctuation, casing, emojis, or negation.
- Ignoring language, dialect, cultural context, and labeler disagreement.
- Evaluating generation only with automatic metrics and no human review.

21. Transformers, Large Language Models, and Generative AI

Attention

Attention allows each token to weigh the relevance of other tokens when building its representation. This helps the model connect words across long distances and process sequences in parallel.

Transformer building blocks

- **Token embeddings:** convert token IDs into vectors.
- **Positional information:** indicates token order.
- **Self-attention:** mixes information across tokens.
- **Feed-forward layers:** transform each token representation.
- **Residual connections:** help information and gradients flow.
- **Layer normalization:** stabilizes activations.

Encoder, decoder, and encoder-decoder models

Architecture	Best known for	Typical tasks
Encoder-only	Understanding representations	Classification, retrieval, token labeling
Decoder-only	Next-token generation	Chat, completion, code generation
Encoder-decoder	Transforming one sequence into another	Translation, summarization

Large language models (LLMs)

An LLM is trained on large text collections to predict tokens. During training it learns patterns of language, knowledge associations, and task-like behavior. It does not retrieve truth automatically and can produce fluent but incorrect output.

Prompting

- Give the model a clear role and task.
- Provide the necessary context, constraints, and output format.
- Include examples when the desired behavior is hard to describe.
- Tell the model what to do when information is missing.
- Test many realistic and adversarial inputs.
- Do not place secrets or private data in third-party prompts without approval.

Fine-tuning vs. prompting vs. RAG

Approach	Changes model weights?	Best for
Prompting	No	Giving instructions and examples quickly
Fine-tuning	Yes	Consistent style, format, or specialized behavior
RAG	No base-model weight change	Grounding responses in current or private documents
Pretraining	Yes, from large corpora	Building a foundation model; usually beyond student resources

Evaluating generative AI

- Factual correctness and source grounding.
- Relevance and completeness.
- Safety and policy compliance.
- Consistency across repeated runs.
- Latency and cost.
- Human preference or task success.
- Performance on edge cases, ambiguous prompts, and prompt injection attempts.

Hallucination: A hallucination is a generated statement not supported by reliable evidence or the provided context. Fluent wording is not proof of correctness.

22. Retrieval-Augmented Generation and Vector Search

What RAG does

Retrieval-augmented generation (RAG) first searches a document collection for relevant passages and then gives those passages to a language model as context. This can improve grounding and make private or current information available without retraining the base model.

Typical RAG pipeline

18. Collect and clean approved documents.
19. Split documents into chunks.
20. Create an embedding for each chunk.
21. Store embeddings with source metadata in a vector index.
22. Embed the user's query.
23. Retrieve the most similar chunks.
24. Optionally rerank the results.
25. Generate an answer using the retrieved context.
26. Return citations or source links.
27. Evaluate retrieval and generation separately.

Chunking

Chunks should be large enough to preserve meaning but small enough for precise retrieval. Fixed-size chunks are simple; structure-aware chunks based on headings, paragraphs, or sections often work better.

Vector search

Vector search compares embeddings using cosine similarity, dot product, or a related measure. Semantic similarity can retrieve passages that use different wording from the query.

RAG failure modes

Failure	Possible cause	Possible fix
Relevant passage not retrieved	Poor chunking or embeddings	Change chunking, query rewriting, or embedding model
Too much irrelevant context	k too large or weak ranking	Reduce k or add reranking
Answer ignores context	Prompt or model behavior	Require context-only answers and abstention
Answer cites wrong passage	Metadata mismatch	Preserve stable source IDs and verify citation mapping
Prompt injection in documents	Untrusted content treated as instruction	Separate instructions from data and sanitize/flag content

Evaluation split: Measure retrieval quality first: did the correct evidence appear in the retrieved set? Then measure answer quality. A generation model cannot use evidence it never received.

23. Responsible AI, Bias, Privacy, and Interpretability

Bias can enter anywhere

- Data collection may exclude groups or reflect historical inequities.
- Labels may be subjective or inconsistent.
- Features may act as proxies for protected characteristics.
- The metric may hide poor performance for smaller groups.
- Deployment may change behavior or create feedback loops.

Fairness checks

When appropriate and legally permitted, compare relevant metrics across groups, examine false-positive and false-negative rates, inspect representation in the dataset, and document uncertainty. Fairness definitions can conflict, so teams should identify the real-world harm they are trying to reduce.

Privacy

- Collect only the data needed for the project.
- Use de-identified or synthetic data when possible.
- Limit access and store data in approved locations.
- Do not expose personal information in notebooks, screenshots, demos, or prompts.
- Understand consent, licenses, terms of use, and data-retention expectations.
- Delete local copies when the project no longer requires them.

Interpretability

Method	What it helps explain	Limitation
Coefficients	Direction and strength in linear models	Depends on scaling and model assumptions
Feature importance	Which features influenced tree splits	May favor high-cardinality or correlated features
Permutation importance	Performance drop when a feature is shuffled	Correlated features complicate interpretation
Partial dependence	Average prediction response to a feature	Can evaluate unrealistic feature combinations
SHAP-like attribution	Local and global contribution estimates	Can be computationally expensive and misread as causality

Interpretation is not causation: A model can reveal associations useful for prediction without proving that changing a feature will cause the predicted outcome to change.

24. Git, GitHub, and Team Collaboration

Core workflow

```
# First time
 git clone <repository-url>
 cd <repository>

# Start work
 git checkout main
 git pull
 git checkout -b feature/data-cleaning

# Save work
 git status
 git add src/clean_data.py
 git commit -m "Add missing-value handling"
 git push -u origin feature/data-cleaning
```

Pull requests

A pull request (PR) asks teammates to review and merge a branch. A good PR explains the goal, the main changes, how it was tested, and any remaining limitations.

Good commit messages

Weak	Better
update	Add TF-IDF baseline for sentiment classification
fix stuff	Fix label encoding for unseen validation classes
final version	Refactor training loop and save best checkpoint

Merge conflicts

A merge conflict occurs when Git cannot automatically combine changes. Read the conflict markers, keep the correct content, remove the markers, test the result, then commit the resolution. Ask a lead before force-pushing shared branches.

Recommended branch rules

- Keep main runnable and stable.
- Use short-lived branches for one feature or fix.
- Open a PR instead of pushing directly to main.
- Request at least one review for meaningful changes.
- Do not commit generated datasets, checkpoints, secrets, or notebook clutter unless the team agreed to do so.
- Pull main before starting and before merging.

Code review checklist

- Does the code solve the stated issue?
- Are names and comments clear?
- Are data assumptions documented?
- Could preprocessing leak information?
- Are edge cases and errors handled?
- Can another member reproduce the result?
- Were tests or validation runs included?

25. Reproducible Experiments and Project Organization

Suggested folder structure

```
project/
├── README.md
├── requirements.txt
├── .gitignore
├── data/
│   ├── raw/          # usually not committed
│   └── processed/    # usually not committed
├── notebooks/
│   ├── 01_eda.ipynb
│   └── 02_baseline.ipynb
├── src/
│   ├── data.py
│   ├── features.py
│   ├── models.py
│   └── evaluate.py
├── configs/
├── tests/
├── reports/
└── models/           # checkpoints, usually external storage
```

Random seeds

Random seeds make many operations repeatable, including splits, initialization, and sampling. Exact reproducibility can still depend on hardware and low-level operations, but seeds improve fair experiment comparisons.

```
import random
import numpy as np
import torch

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

Experiment log

Field	Example
Experiment ID	cnn_014
Goal	Test stronger augmentation
Data version	images_v3
Split	grouped by user, seed 42
Model	pretrained CNN
Hyperparameters	lr=1e-4, batch=32, epochs=20
Metric	macro F1
Result	0.842 validation
Notes	Improved minority class recall; slower training

README essentials

- Project purpose and team members.
- Dataset source, access instructions, and license notes.
- Environment setup.
- How to run preprocessing, training, evaluation, and the demo.
- Current results and baseline.
- Known limitations and next steps.
- Where large artifacts are stored.

26. Deployment, APIs, and Model Serving

What deployment means

Deployment makes a model available to users or another software system. A model can be deployed as a web API, an application feature, a batch job, an on-device model, or an internal analysis tool.

Inference pipeline

28. Receive an input.
29. Validate the input format.
30. Apply the exact training preprocessing.
31. Run the model in evaluation mode.
32. Postprocess outputs into a useful format.
33. Log safe operational information.
34. Return the result and handle errors gracefully.

Simple API idea

```
# Conceptual FastAPI-style example
from fastapi import FastAPI

app = FastAPI()

@app.post("/predict")
def predict(request: PredictionRequest):
    features = preprocess(request)
    prediction = model.predict(features)
    return {"prediction": prediction.tolist()}
```

Deployment concerns

Concern	Question
Latency	How quickly must the answer return?
Throughput	How many requests arrive at once?
Cost	What compute or API usage is required?
Reliability	What happens when a dependency fails?
Security	Can inputs expose secrets or attack the system?
Monitoring	How will the team detect performance changes?
Versioning	Which model and preprocessing version produced the result?

Data drift and concept drift

Data drift means the input distribution changes. Concept drift means the relationship between inputs and outcomes changes. Both can reduce performance after deployment, even if the code remains unchanged.

27. Debugging and Troubleshooting

Start with the simplest checks

35. Read the full error message from the bottom upward.
36. Print shapes, types, ranges, and a few example values.
37. Confirm the correct environment and package imports.
38. Run the smallest failing example.
39. Check whether preprocessing is identical across splits.
40. Compare your assumptions with the actual dataset.
41. Use version control to find the change that introduced the problem.

Model is not learning

- Labels may be wrong, shuffled, or the wrong data type.
- Learning rate may be too small or too large.
- Loss function may not match the output and label format.
- Gradients may be disabled, zero, exploding, or never updated.
- Inputs may be constant, incorrectly normalized, or full of missing values.
- The model may be too simple—or the task may contain little predictive signal.
- Try overfitting a tiny batch. Failure suggests an implementation problem.

Training loss decreases but validation does not

- Overfitting.
- Train and validation distributions differ.
- Data leakage made training artificially easy.
- Validation labels or preprocessing are wrong.
- Metric implementation differs between training and validation.
- The split contains group, time, or duplicate problems.

NaN loss

- Lower the learning rate.
- Inspect input values for NaN, infinity, or extreme scale.

- Use numerically stable loss functions provided by the framework.
- Clip gradients if they explode.
- Check illegal operations such as log(0) or division by zero.
- Confirm mixed-precision settings are safe.

Shape mismatch

Write down the expected shape after every major layer. Print tensor.shape during a small forward pass. Remember that image libraries may use different channel orders and classification labels usually have shape (batch,), not (batch, 1), for standard cross-entropy loss.

Debugging principle: Change one thing at a time. Record the result. Randomly changing five hyperparameters makes it impossible to know what fixed—or broke—the system.

28. Team Project Checklists

Project kickoff checklist

- Problem statement and intended user are clear.
- Success metric and baseline are defined.
- Dataset source, permissions, and limitations are documented.
- Team roles and communication channels are assigned.
- GitHub repository and folder structure exist.
- Milestones are small enough to demonstrate progress early.
- Risks, privacy concerns, and ethical questions are discussed.

Before training

- Each row/example has a clear meaning.
- Duplicates and group relationships are understood.
- Splits are created before learned preprocessing.
- The target distribution is inspected.
- A simple baseline is implemented.
- The primary metric matches the project goal.
- Inputs and labels pass sanity checks.

Before presenting results

- Report performance on unseen data.
- Compare with the baseline.
- Include the metric definition and why it was chosen.
- Show representative successes and failures.
- Avoid claiming causation from prediction.
- Disclose dataset and model limitations.
- Ensure the demo does not expose personal or secret data.
- Make the repository runnable for another team member.

Before merging a pull request

- Pull the latest main branch and resolve conflicts.
- Run the relevant code or tests.
- Remove debugging prints and unused files.
- Update documentation and requirements when needed.
- Check that no secrets, large files, or private data were added.
- Write a clear PR summary and request review.

29. Model Selection Cheat Sheet

Situation	Good first model	Why
Tabular binary classification	Logistic regression, random forest, gradient boosting	Strong baselines and complementary assumptions
Tabular regression	Ridge regression, random forest, gradient boosting	Linear and nonlinear comparisons
Sparse text classification	TF-IDF + logistic regression or linear SVM	Fast and difficult to beat on small/medium data
Small labeled image dataset	Pretrained CNN with transfer learning	Uses learned visual features
Image classification from scratch	Small CNN	Useful for learning; needs enough data
Semantic text similarity	Sentence embeddings + cosine similarity	Captures meaning beyond exact words
Document question answering	RAG pipeline	Grounds answers in retrieved text
Clustering scaled numeric data	k-means, hierarchical, DBSCAN	Compare assumptions and inspect clusters
Need interpretable rules	Shallow decision tree or linear model	Human-readable structure
Very imbalanced classification	Class-weighted baseline + PR-AUC/F1/recall	Focuses evaluation on rare positives

When not to use deep learning

- The dataset is small and structured.
- A simpler model already meets the goal.
- Interpretability or low latency is essential.
- The team cannot support the compute, tuning, or maintenance.
- The data pipeline is not yet reliable.
- There is no clear evaluation plan.

Hyperparameter tuning priorities

Model family	Start by tuning
Logistic/ridge/lasso	Regularization strength, class weights
kNN	Number of neighbors, distance metric, scaling
SVM	C, kernel, gamma, scaling
Decision tree	Depth, minimum leaf size
Random forest	Number of trees, depth, leaf size, max features
Gradient boosting	Learning rate, number of trees, tree depth, subsampling
Neural network	Learning rate, architecture size, batch size, regularization, epochs

30. Glossary of Common AI Terms

Term	Beginner-friendly definition
Activation function	Nonlinear function applied inside a neural network.
API	Interface that lets software systems communicate.
Backpropagation	Method for computing neural-network gradients.

Term	Beginner-friendly definition
Baseline	Simple reference method a model should beat.
Batch	Subset of examples processed together.
Checkpoint	Saved model state during training.
Class imbalance	Some labels occur much more often than others.
Classification	Predicting a category.
Confusion matrix	Table of actual and predicted classes.
Cross-validation	Repeated train/validation splitting for stable evaluation.
Data leakage	Training uses information unavailable at real prediction time.
Data drift	Input data distribution changes after deployment.
Embedding	Dense vector representation of meaning or characteristics.
Epoch	One full pass through the training dataset.
Feature	Input variable used by a model.
Feature engineering	Creating useful model inputs from raw data.
Fine-tuning	Continuing training of a pretrained model for a new task.
Gradient	Vector of derivatives showing how loss changes with parameters.
Gradient descent	Optimization method that updates parameters against the gradient.
Hyperparameter	Setting chosen outside the model's learned parameters.
Inference	Using a trained model to make predictions.
Label/target	The answer a supervised model learns to predict.
Learning rate	Step size used by an optimizer.
Logit	Raw classification score before probability conversion.
Loss function	Number measuring model error during training.
Model	Learned mathematical mapping from inputs to outputs.
Overfitting	Learning training noise instead of generalizable patterns.
Parameter	Value learned from training data.
Pipeline	Ordered sequence of preprocessing and modeling steps.
Pretraining	Training on a broad dataset before adaptation.
Regression	Predicting a continuous number.
Regularization	Methods that discourage excessive model complexity.
RAG	Retrieval-augmented generation using external evidence.
Tensor	Multidimensional numerical array used in deep learning.
Token	Text unit processed by an NLP model.
Train/validation/test split	Separate data for fitting, selection, and final evaluation.
Transfer learning	Adapting knowledge from a pretrained model.
Underfitting	Model is too simple or insufficiently trained.
Vector database/index	System optimized for storing and searching embeddings.

31. Recommended Learning Path for New Members

Stage 1: Be able to contribute to data and code

- Python variables, functions, loops, lists, dictionaries, and errors.
- pandas loading, filtering, grouping, missing values, and saving files.
- NumPy shapes and basic array operations.
- Git clone, branch, commit, pull, push, and pull requests.
- Basic visualizations and dataset sanity checks.

Stage 2: Understand the modeling workflow

- Features, targets, classification, and regression.
- Train/validation/test splits and data leakage.
- Baselines, preprocessing pipelines, and cross-validation.
- Accuracy, precision, recall, F1, MAE, RMSE, and error analysis.
- Overfitting, regularization, and model comparison.

Stage 3: Learn the project's model family

- Tabular project: linear models, trees, forests, and boosting.
- Image project: tensors, CNNs, augmentation, and transfer learning.
- Text project: TF-IDF, embeddings, transformers, and evaluation.
- Generative-AI project: prompting, RAG, retrieval evaluation, and safety.

Stage 4: Become an independent project contributor

- Turn an idea into a testable experiment.
- Write reusable functions and maintainable modules.
- Review code and explain technical decisions.
- Diagnose model and data problems systematically.
- Communicate results honestly, including limitations.
- Help newer members understand the workflow.

What project leads should reinforce

- Beginners should receive small, well-defined tasks with examples.
- Every technical choice should connect to the project goal.
- Data quality and evaluation matter more than flashy model names.
- Team members should document decisions and share knowledge.
- A safe, inclusive environment makes it easier to ask questions and catch mistakes.

Final reminder: A successful ACM AI project is not just a high metric. It is a well-framed, reproducible, ethical project that teaches team members how to reason about data, models, software, and real users.

Appendix A: One-Page ML Workflow Quick Reference

Step	Key question	Output
1. Frame	What decision or user need are we supporting?	Problem statement
2. Inspect	What does the data contain and what can go wrong?	EDA notes
3. Split	How do we prevent leakage and represent future use?	Train/validation/test sets
4. Baseline	What simple method must we beat?	Baseline score
5. Preprocess	How are missing, categorical, scaled, text, or image inputs handled?	Reproducible pipeline
6. Train	Which models fit the data and constraints?	Candidate models
7. Evaluate	Which metric represents success?	Metrics and confidence
8. Analyze	Where and why does the model fail?	Error categories
9. Iterate	Would better data, features, tuning, or modeling help most?	Next experiment
10. Communicate	Can others reproduce and trust the result?	README, report, demo

Appendix B: Neural Network Quick Reference

Concept	Meaning
Forward pass	Inputs move through layers to produce predictions
Loss	Measures prediction error
Backward pass	Computes gradients using backpropagation
Optimizer step	Updates parameters
Learning rate	Controls update size
Batch size	Examples per update
Epoch	One pass through training data
Dropout	Randomly disables activations during training
Normalization	Stabilizes activation distributions
Early stopping	Stops when validation performance no longer improves

Appendix C: Questions to Ask Before Using a Model

- What baseline does this model improve on?
- What assumptions does it make?
- How much data and computing does it need?
- What metric and threshold will we use?
- How will we detect leakage?
- How will we explain failures?
- What groups or situations may be underserved?
- Can another team member reproduce the result?
- What happens when the model is uncertain?

- What will we monitor after deployment?